Where Should Resolution Be Preserved? Slicing, Tiled Inference and Heterogeneous Box Fusion for Small Bird Detection

Vũ Quốc Đạt[1], Nguyễn Gia Phúc[1], Ngô Ngọc Trường Sơn[1*], Phan Thị Thu Hồng[1]

[1*]FPT University, Da Nang, Vietnam.

*Corresponding author(s). E-mail(s): truongsonks2006@gmail.com;
Contributing authors: qd130506@gmail.com; phucoik@gmail.com;
[email];

Abstract

Birds in the MVA 2023 Small Object Detection Challenge for Spotting Birds (SOD4SB) occupy a median of 16 pixels in 3840×2160 frames, which makes every choice that touches image resolution decisive. We study where resolution should be preserved: during training, during inference, or in the fusion of complementary detectors. Starting from a YOLOv8-M detector pretrained on Drone2021 and fine-tuned on native-resolution crops, we compare three training-time data strategies (colour augmentation, copy-paste and exhaustive tile sampling, "slicing"), two inference-time strategies (horizontal-flip test-time augmentation and SAHI tiled inference), several box merging rules, and a heterogeneous ensemble with a GFL ResNet-50 FPN detector. Slicing gives the largest single gain (70.2% to 79.3% AP50), whereas SAHI *reduces* accuracy (79.3% to 67.5%) when full frames are already processed without downscaling; we trace the loss to scale mismatch, tile-border truncation and over-confident empty tiles, and show that correcting them recovers but does not exceed plain inference. Weighted boxes fusion (WBF) of flipped and unflipped predictions reaches 81.0%, and a single WBF over YOLO, flipped YOLO and GFL reaches 84.5% AP50, 45.8% AP75 and 53.4% AR100 on the validation split. A scale-wise analysis explains the gain: YOLO is stronger on 8–16 px birds and GFL on 32–64 px birds. Merging rules and ensemble choices are checked on held-out halves of the validation set, and the code and results are publicly available.

Keywords: small object detection, bird detection, slicing, test-time augmentation, weighted boxes fusion, ensemble

1 Introduction

Detecting birds in high-resolution video is needed for drone safety, airport bird-strike prevention and wind farm monitoring [1–3]. The SOD4SB challenge [4] formalises this task: 4K frames, a single *bird* class, and objects that are mostly smaller than 32 pixels. At this scale the detector sees only a few pixels of texture, birds are easily confused with leaves, wires and compression noise, and a one-pixel localisation error changes the intersection over union (IoU) noticeably.

Two families of remedies are common in small object detection. *Training-time* remedies change the data the detector learns from, for example cropping high-resolution images instead of shrinking them, or pasting extra objects [5]. *Inference-time* remedies change how predictions are produced, for example slicing-aided hyper inference (SAHI) [6], test-time augmentation (TTA) and model ensembling [7, 8]. These remedies are often combined in challenge solutions, but it is rarely shown which of them are complementary and which are redundant under a given pipeline.

This paper answers that question with controlled ablations on SOD4SB, and Section 5.4 reports how the study was carried out on a single laptop GPU, including the problems met on the way. Our main contributions are:

- We compare colour augmentation, copy-paste and exhaustive tile sampling (slicing) on the same YOLOv8-M [9] initialisation and schedule, and find that only slicing helps (+9.1 AP50).
- We show that SAHI degrades a detector that already processes full frames at native resolution, identify three causes (scale mismatch, truncated boxes at tile borders, over-confident empty tiles), and quantify how much each correction recovers.
- We show that a single weighted boxes fusion of a slicing-trained YOLOv8-M, its flipped view and a GFL detector [10] raises AP50 from 79.3% to 84.5%, and explain the gain with a scale-wise analysis in which the two architectures are strong on different bird sizes.
- We report an evaluation protocol that guards against selection bias (held-out halves of the validation set) and release code, configurations and per-image results.

The rest of the paper is organized as follows. Section 2 reviews related works in the literature. Section 3 describes the detectors, the training- and inference-time strategies and the box fusion. Section 4 presents the datasets and the experimental setting. Section 5 reports the experimental results, the scale-wise and error analyses, a comparison with a published reference, the development process and the limitations. Finally, Section 6 concludes the paper and outlines future research directions.

2 Related Works

Bird datasets include wind-farm time-lapse images [3], airport surveillance (AirBirds) [2] and drone-mounted cameras (Drone2021) [1], which also serves as a pretraining source in SOD4SB [4]. General detectors such as Faster R-CNN [11], Cascade R-CNN [12], CenterNet [13], GFL [10] and VarifocalNet [14] were designed on COCO [15], where tiny objects are rare. Tiny-object work therefore revisits

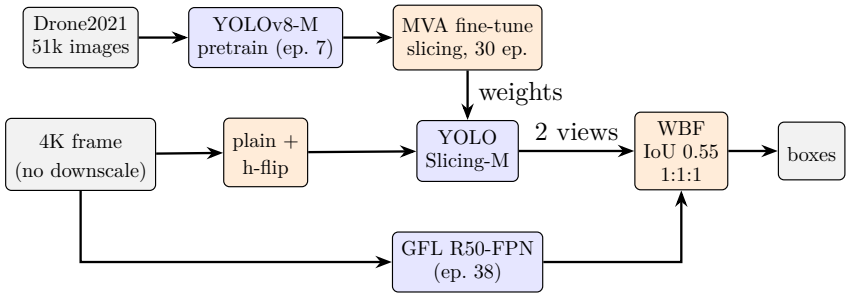


Fig. 1 Final pipeline. A slicing-trained YOLOv8-M predicts on the full native-resolution frame and on its horizontal mirror; a GFL ResNet-50 FPN predicts on the same frame. One weighted boxes fusion merges the three sources with equal weights.

matching and metrics, e.g. the normalised Wasserstein distance [16], universal-scale benchmarks [17] and scale-wise evaluation [18].

SAHI [6] fine-tunes on slices and, at test time, predicts on overlapping tiles that are upsampled to the network input, merging the results with the full-image prediction. Its premise is that the full image would otherwise be downscaled. We revisit this premise for a pipeline that never downscales.

Flip TTA and multi-model ensembles are standard in challenges. Non-maximum suppression (NMS) keeps one box per cluster, whereas weighted boxes fusion (WBF) [7] averages the coordinates of all members. The SOD4SB winning solutions combined several detectors [4, 8]; we isolate the contribution of architectural diversity with a scale-wise analysis.

Despite these advances, existing studies rarely examine, under one controlled protocol, at which stage of the pipeline resolution should be preserved for tiny objects in 4K frames: in the training samples, in the test-time views, or through the fusion of heterogeneous detectors. In particular, the benefit of SAHI is usually reported for detectors that downscale their input, and the complementarity of heterogeneous detectors is seldom explained by object size. This motivates the present study, which evaluates these stages separately on SOD4SB and combines only the components that help.

3 Methodology

3.1 Overview of the Proposed Approach

Fig. 1 illustrates the overall workflow. The approach consists of two phases: training two heterogeneous detectors with resolution-preserving data strategies, and combining their predictions on several views of the native-resolution frame by box fusion. Each component was retained only if it improved AP50 in a controlled ablation; components that were tried and rejected (colour augmentation, copy-paste, SAHI) are analysed in Section 5.

Let $I \in \{0, \dots, 255\}^{H \times W \times 3}$ be a video frame with $H = 2160$ and $W = 3840$, and let $\mathcal{G} = \{g_j\}_{j=1}^{n}$ be its annotated birds, each an axis-aligned box $g_j = (x_1, y_1, x_2, y_2)$. A detector f maps the frame to a set of scored boxes $\mathcal{B} = f(I) = \{(b_i, s_i)\}$ with

$b_i \in \mathbb{R}^4$ and $s_i \in [0, 1]$. A prediction is correct at threshold t if its intersection over union with an unmatched annotated box, $\mathrm{IoU}(b, g) = |b \cap g|/|b \cup g|$, is at least t. Birds have a median $\sqrt{\text{area}}$ of 16 pixels, less than 0.5% of the frame width, so any operation that rescales I changes what the detector can see.

Resolution can be preserved, or lost, at three places: in the training samples from which f is learned, in the views v on which it predicts at test time, with $\mathcal{B}_v = v^{-1}\big(f(v(I))\big)$ mapped back to frame coordinates, and in the fusion Φ of several sources. With a YOLOv8-M detector f_Y, a GFL detector f_G and the horizontal flip h, the final system is

$$\hat{\mathcal{B}} = \Phi\Big(f_Y(I),\ h^{-1}\big(f_Y(h(I))\big),\ f_G(I)\Big). \tag{1}$$

3.2 Candidate Detector Architectures

Two detectors with different designs and input resolutions are used, so that their errors are as little correlated as possible. Both are trained without any downloaded weights: they are first pretrained on Drone2021 and then fine-tuned on MVA.

3.2.1 YOLOv8-M

We use the Ultralytics YOLOv8-M architecture [9] with one class. It is trained from random initialisation on Drone2021 for 10 epochs; the epoch-7 checkpoint, which has the highest Drone2021 validation AP50 (44.6%), initialises every MVA branch. Each training sample is a native-resolution 1280×736 crop: no resizing is applied, so a 12-pixel bird remains 12 pixels.

3.2.2 GFL R50-FPN

The second detector is Generalized Focal Loss (GFL) [10] with a ResNet-50 [19] and FPN [20], implemented with UniverseNet [17]. It is trained from random initialisation on Drone2021 for 6 epochs and then on MVA, with input 2560×1440 (a 2/3 downscale of the 4K frame), batch size one with gradient accumulation over eight images, and per-image batch normalisation, which is used identically in training and inference. Of the 50 MVA epochs we use epoch 38, the one with the highest validation AP50 (Section 4.2).

3.3 Training-Time Data Strategies

All YOLO branches share the same initial weights, optimiser, schedule, batch size and a common augmentation (horizontal flip with $p = 0.5$ and brightness scaling in $[0.8, 1.2]$). They differ only in how crops are produced:

- **Baseline:** one bird-centred native crop per image per epoch.
- **Colour augmentation:** the baseline plus, with $p = 0.8$, a hue shift of ± 5, saturation and gamma scaling in $[0.8, 1.2]$.
- **Copy-paste:** the baseline plus, with $p = 0.3$, up to three bird patches (from a bank of 24,485 training patches) pasted at random positions [5].

- **Slicing:** every tile of a 25%-overlap grid of 1280×736 windows that contains a bird, plus one bird-free tile per image, is used in every epoch. This yields 31,790 samples per epoch instead of 8,880.

3.4 Inference-Time Strategies

YOLO predictions are made on the full 3840×2160 frame padded to a multiple of 32, with confidence threshold 0.001 and at most 300 boxes per image.

Flip TTA. The frame and its horizontal mirror are predicted; boxes of the mirrored view are mapped back by $x' = W - x$.

SAHI. In addition to the full frame, windows of size $1280/z \times 736/z$ with 25% overlap are cropped and resized to 1280×736, where z is the zoom factor. For $z = 1.5$ this gives 36 tiles per frame; for $z = 1$, 16 tiles. We evaluate three corrections: (i) discarding tile boxes that touch an inner tile border (truncated birds), (ii) multiplying tile scores by a factor $\alpha < 1$, and (iii) keeping a tile box only if it overlaps a full-frame box with score ≥ 0.05.

3.5 Heterogeneous Weighted Boxes Fusion

The fusion Φ in (1) is weighted boxes fusion (WBF). Let $\mathcal{B} = \bigcup_{m=1}^{M} \mathcal{B}_m$ be the boxes of M sources (models or views). WBF [7] visits boxes in decreasing score order and assigns each to the first cluster whose fused box has IoU above τ, creating a new cluster otherwise. For a cluster C with members (b_i, s_i), the fused box and score are

$$\hat{b} = \frac{\sum_{i \in C} s_i \, w_{m(i)} \, b_i}{\sum_{i \in C} s_i \, w_{m(i)}}, \qquad \hat{s} = \frac{\sum_{m=1}^{M} w_m \max_{i \in C \cap \mathcal{B}_m} s_i}{\sum_{m=1}^{M} w_m}, \tag{2}$$

where w_m is the weight of source m and the maximum over an empty set is zero. A box found by only one of three equally weighted sources therefore keeps one third of its score, which pushes isolated false positives down the ranking. The final system uses $M = 3$ sources (YOLO, flipped YOLO, GFL), $w_m = 1$ and $\tau = 0.55$.

3.6 Scale-Wise and Error Analysis

To understand why components help, predictions are also analysed by bird size and by error type. Annotated boxes are grouped into buckets of $\sqrt{\text{area}}$ (8–16, 16–32 and 32–64 pixels), and AP50 is computed per bucket by restricting the COCO evaluation to that area range. In addition, each model is examined at its best-F1 operating point at IoU 0.5, where the numbers of false positives (FP) and false negatives (FN) are counted, and the missed birds are cropped and inspected visually.

Table 1 Datasets used in this work.

Split	Images	Boxes	Use
Drone2021 train [1]	51,119	55,469	pretraining
Drone2021 val	13,572	11,197	checkpoint selection
MVA train [4]	8,880	26,172	fine-tuning
MVA val (audited)	879	2,864	all ablations
MVA public test	9,699	–	submission only

4 Experiments

4.1 Data Description

To train and evaluate the detectors, two bird datasets are used. MVA 2023 SOD4SB is the primary benchmark, and Drone2021 serves as the pretraining source for both detectors. Table 1 lists the splits.

4.1.1 MVA 2023 SOD4SB Dataset

The SOD4SB challenge dataset [4] consists of 4K (3840×2160) video frames with a single *bird* class; the median bird measures 16 pixels in $\sqrt{\text{area}}$. We train on the official training split and report results on the official validation split after removing one exactly duplicated box (2,864 instead of 2,865 boxes; scores change by less than 0.1 point). The public test set (9,699 frames) has no released labels; we use it only to produce a single submission with the configuration selected on validation.

4.1.2 Drone2021 Dataset

Drone2021 [1] contains images of distant birds taken from drone-mounted cameras. Its birds are larger than those of MVA (median $\sqrt{\text{area}}$ of 25–31 pixels) and its boxes follow a different annotation convention, so it is used only for pretraining and for selecting the pretraining checkpoint.

4.2 Experimental Setting

All experiments were conducted on one NVIDIA RTX 5070 Laptop GPU (8 GB) with seed 42. YOLO uses the Ultralytics model definition with our own PyTorch training loop, and GFL is implemented with UniverseNet (MMDetection). Neither detector uses ImageNet or COCO weights. YOLO branches are trained with AdamW [21] (no decay on biases and normalisation), a cosine learning rate after 100 warm-up updates, BF16 forward passes with FP32 loss and parameters, and exponential moving average weights for evaluation. GFL uses SGD (momentum 0.9, weight decay 10^{-4}) and FP16 mixed precision. The two-stage training configurations are summarised in Table 2.

Evaluation metrics. We use the COCO [15] evaluation with at most 100 detections per image and report three scores. AP_t is the area under the precision–recall curve obtained by ranking all predictions by score, where a prediction is a true positive if its IoU with a still unmatched annotated box is at least t. We report AP50

Table 2 Two-stage training configuration of each detector.

	Stage 1: Drone2021 pretraining	Stage 2: MVA fine-tuning
YOLOv8-M		
Input	native 1280×736 crops (no resizing)	
Augment.	h-flip p=0.5, brightness [0.8, 1.2]	
Epochs	10 (epoch 7 kept)	30 (last epoch)
Batch size	6	6
Optimiser, LR	AdamW, $10^{-3} \rightarrow 10^{-4}$	AdamW, $10^{-4} \rightarrow 10^{-5}$
GFL R50-FPN		
Input	full frame resized to 2560×1440	
Augment.	h-flip p=0.5	
Epochs	6	50 (epoch 38 kept)
Batch size	1 (×8 accumulation)	1 (×8 accumulation)
Optimiser, LR	SGD, 0.005, steps at 4, 5	SGD, 0.001, steps at 35, 45

($t = 0.5$), the main metric of the challenge, and AP75 ($t = 0.75$), which rewards tight boxes. AR100 measures how many birds are found at all:

$$\mathrm{AR100} = \frac{1}{10} \sum_{t \in \{0.50, 0.55, \ldots, 0.95\}} \mathrm{Recall}_t, \tag{3}$$

where Recall_t is the fraction of annotated birds matched at threshold t by the top 100 predictions of their frame. For the scale-wise analysis we also report AP50 per bucket of $\sqrt{\mathrm{area}}$. All YOLO results on MVA use the last (30th) epoch. The GFL checkpoint (epoch 38 of 50) was chosen by MVA validation AP50; its last epoch scores 74.2% instead of 74.9%, so GFL results are optimistic by about 0.7 points.

5 Results and Discussion

5.1 Results on the MVA Validation Set

5.1.1 Performance of Individual Detectors

The first two rows of Table 3 compare the two detectors trained without any downloaded weights. GFL reaches 74.9% AP50 and the YOLOv8-M baseline 70.2%; GFL is also ahead in AP75 (36.3% vs. 25.5%) and AR100 (46.6% vs. 43.9%), although it sees a frame downscaled to 2/3. Training YOLOv8-M with slicing reverses the order: it reaches 79.3% AP50, 38.8% AP75 and 50.0% AR100 and becomes the strongest single model. The two detectors take a similar time per 4K frame (294 and 337 ms).

Table 3 Step-by-step results on MVA validation (%). Time is per 4K frame on an RTX 5070 Laptop GPU.

Method	AP50	AP75	AR100	ms
GFL R50-FPN	74.9	36.3	46.6	294
YOLOv8-M baseline	70.2	25.5	43.9	338
+ slicing	79.3	38.8	50.0	337
+ flip TTA (WBF)	81.0	40.3	51.1	709
+ GFL, one-stage WBF	**84.5**	**45.8**	**53.4**	≈1003

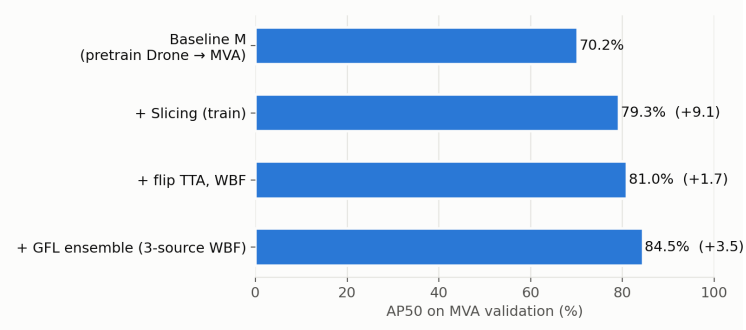


Fig. 2 AP50 after each step of the final pipeline.

5.1.2 Performance of the Proposed Pipeline

Table 3 traces the final pipeline. Slicing contributes the largest gain, flip TTA with WBF adds 1.7 points, and the heterogeneous ensemble adds 3.5 points. AP75 grows from 25.5% to 45.8% and AR100 from 43.9% to 53.4%, i.e. the pipeline improves localisation as well as recall. Fig. 2 shows the same progression.

5.1.3 Training-Time Ablation

Table 4 compares the data strategies. Colour augmentation does not help (−0.2): the frames already span a wide range of illumination. Copy-paste hurts (−3.3), presumably because pasted patches have inconsistent blur, lighting and context with the target background, which teaches the detector shortcuts that do not transfer. Slicing improves every metric, most strongly AP75 (+13.3); note that it also uses 3.6 times more samples per epoch (Section 5.5).

Pretraining alone. The Drone2021 model scores 0.1% AP50 on MVA without fine-tuning, yet its most confident false positives lie on real birds (Fig. 3): its boxes are much larger than the MVA annotations (median $\sqrt{\text{area}}$ of 40 px for predictions with score above 0.3, against 16 px for MVA boxes and 25–31 px for Drone2021 boxes). Fine-tuning therefore mainly adapts the box convention, not the notion of a bird.

Table 4 Training-time strategies (YOLOv8-M, 30 epochs, %).

Strategy	AP50	AP75	AR100
YOLOv8-M baseline	70.2	25.5	43.9
+ colour aug.	70.0	25.5	43.8
+ copy-paste	66.9	23.2	42.2
+ slicing	**79.3**	**38.8**	**50.0**
Drone2021 pretrain only	0.1	0.0	2.5

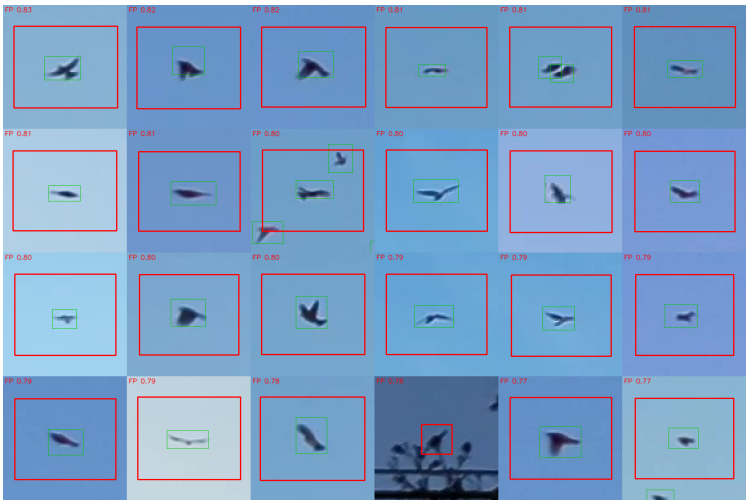


Fig. 3 Most confident false positives of the Drone2021-only model on MVA (red: prediction, green: ground truth). The birds are found, but the boxes are too large to reach IoU 0.5.

5.1.4 Inference-Time Ablation and the Failure of SAHI

Table 5 reports inference variants on the slicing model. Flip TTA gives a small, consistent gain. SAHI with $z = 1.5$ loses 11.8 AP50 under NMS, and AP75 drops far more (38.8% to 28.7%) than AR100 (50.0% to 47.4%): the birds are still found, but many of their boxes become imprecise. Three mechanisms explain the loss. First, *no information is added*: the full frame is already processed at native resolution, so a zoomed tile only interpolates existing pixels. Second, *scale mismatch*: zooming by 1.5 turns 32–64 px birds into 48–96 px birds, a size rarely seen in training. Third, *tile artefacts*: birds cut by a tile border produce partial boxes with low IoU to the full-frame box, which NMS does not suppress, and bird-free tiles produce confident false positives (at the best F1 operating point, false positives rise from 224 to 786). Discarding border-touching boxes and halving tile scores recover 10.2 points; with $z = 1$ the tiles match the training distribution and SAHI reaches, but does not exceed, plain inference while being three times slower.

Table 5 Inference variants of the slicing model (%). "Fixed" is the best correction of Section 3.4; tiles per frame in brackets.

Inference	Merge	AP50	AP75	AR100	ms
Full frame	–	79.3	38.8	50.0	337
Flip TTA	NMS 0.6	79.9	38.5	50.6	709
	WBF	**81.0**	**40.3**	**51.1**	709
SAHI, $z = 1.5$ (36)	NMS 0.6	67.5	28.7	47.4	1748
	fixed	77.7	38.3	49.3	1748
SAHI, $z = 1.5$ + flip (72)	NMS 0.6	66.0	27.5	47.9	3621
	fixed	78.2	38.0	49.6	3621
SAHI, $z = 1.0$ (16)	NMS 0.6	78.5	38.0	50.2	963
	fixed	79.3	38.6	49.5	963

Table 6 Fusion of YOLO (plain + flip) and GFL (%).

Fusion	AP50	AP75	AR100
TTA by NMS → WBF with GFL	84.4	45.2	52.9
TTA by WBF → WBF, w_{YOLO}=1.5	84.5	45.3	53.3
One-stage WBF, 1:1:1	**84.5**	**45.8**	**53.4**

5.1.5 Configuration Ablation of the Fusion

Table 6 compares ways of combining the two YOLO views with GFL. All three reach 84.4–84.5% AP50; the one-stage fusion has the best AP75 and AR100 and needs no tuned weights, so it is used for the submission.

5.1.6 Scale-Wise Analysis of the Ensemble

Table 7 and Fig. 4 split AP50 by bird size. YOLO is clearly better on 8–16 px birds (80.2% vs. 67.4%), which make up half of the validation set; a likely reason is that YOLO sees them at native resolution, whereas GFL works on a frame downscaled to 2/3. GFL is clearly better on 32–64 px birds (72.4% vs. 60.6%); we hypothesise that the downscaled input gives each feature more context, which helps against the camouflage that dominates errors at this size (Section 5.1.7), but we have not isolated this effect. The ensemble inherits the stronger model in each bucket and reduces false negatives from 698 to 611 at the best F1 operating point.

5.1.7 Error Analysis

Fig. 5 shows false negatives of the YOLO model. Most misses are birds camouflaged in grass or foliage, birds on wires and poles, and birds against dark or over-exposed backgrounds; a few are barely visible even to a human annotator. These are failures of appearance rather than of resolution, which is consistent with the plateau of the 32–64 px bucket across all YOLO variants.

Table 7 AP50 (%) by $\sqrt{\text{area}}$, overall AR100 (%) and best-F1 operating point. Buckets with fewer than 20 birds are omitted.

Model	8–16 (1,456)	16–32 (1,070)	32–64 (316)	AR100	F1	FP/FN
YOLO baseline	64.3	80.3	58.9	43.9	71.8	495/984
YOLO slicing	78.4	86.1	58.4	50.0	81.8	224/728
YOLO slic. + TTA	80.2	87.6	60.6	51.1	82.9	198/698
GFL	67.4	87.7	**72.4**	46.6	77.4	216/920
Ensemble	**83.5**	**89.4**	**72.4**	**53.4**	**84.5**	218/**611**



Fig. 4 AP50 by bird size. YOLO and GFL are strong on different sizes; the ensemble keeps the better of the two.

5.2 Validation on Held-Out Halves

Every choice made on the validation set (checkpoint, inference mode, merging rule and ensemble weights) can overfit it. For merging rules and ensembles we therefore also select on odd image indices and report on even ones, and vice versa. YOLO checkpoints are not selected (last epoch); the GFL checkpoint is the one exception and is not covered by this check. The held-out check confirms the gain of the fusion: choosing the rule on one half of the validation set and scoring the other gives 84.1% and 84.8% AP50 for the ensemble, against 80.7% and 81.4% for flip TTA alone. The improvement of about 3.4 points is therefore not an artefact of selecting the merging rule on the same images.

5.3 Comparison with a Published Reference

Table 8 compares the proposed pipeline with the GFL checkpoint released with the challenge baseline [4, 17], evaluated with our code on the same 879 validation frames and audited labels. The reference is initialised from COCO-pretrained weights and

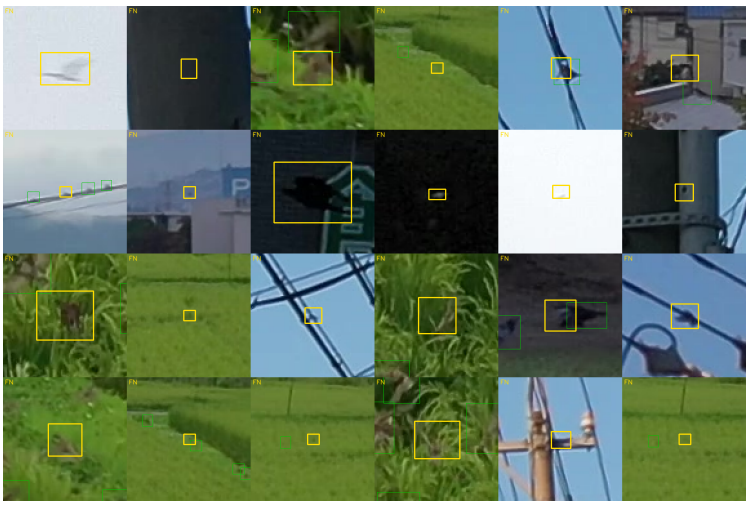


Fig. 5 False negatives of YOLO slicing + TTA (yellow: missed bird, green: other annotated birds). Crops are 96×96 px enlarged three times.

Table 8 Comparison with the released challenge-baseline GFL on MVA validation (%).

Method	Init. / input	AP50	AP75	AR100
Released GFL + flip [4]	COCO / 3840	**88.9**	44.2	**54.2**
Ours: GFL R50-FPN	none / 2560	74.9	36.3	46.6
Ours: YOLO slicing	none / 3840	79.3	38.8	50.0
Ours: one-stage WBF	none / both	84.5	**45.8**	53.4

processes full 3840×2160 frames with flip TTA, whereas our detectors use no downloaded weights. Under this handicap the proposed pipeline is 4.4 points lower in AP50 and 0.8 points lower in AR100, but 1.6 points higher in AP75, i.e. the boxes it finds are more tightly localised. By size, the gap in AP50 is largest for 32–64 px birds (72.4% vs. 83.4%), where the reference benefits from its larger input and its pretraining, and smallest for 8–16 px birds (83.5% vs. 86.8%), where preserving the native resolution matters most. Results of other challenge entries are reported on the public and private test sets with different pretraining and are therefore not directly comparable.

5.4 Development Process and Practical Challenges

The project ran for about seven weeks, from mid-September to 6 November 2026, on a single laptop GPU (RTX 5070 Laptop, 8 GB, Blackwell architecture). Table 9 summarises the phases. This subsection reports the route that led to the final system, including the branches that were abandoned, because several of the design choices of Section 3 were forced by these constraints rather than chosen freely.

Table 9 Project timeline (2026). AP50 on the validation split of the dataset named in each row.

Period	Work	Outcome
16–27 Sep	GPU environment; released GFL checkpoint re-evaluated; GFL from scratch on MVA only	88.9% (released); 6.1% after 1 epoch
27 Sep–7 Oct	GFL: Drone2021 6 epochs, then MVA 50 epochs with per-image BN	74.9% (MVA)
9–28 Oct	First round of YOLO and Swin experiments (batch of one); discarded on 29 Oct	not used
29–31 Oct	YOLOv8-S on Drone2021: BN failure diagnosed, crop recipe rewritten	47.4% (Drone)
31 Oct–1 Nov	YOLOv8-S on MVA (colour augmentation); switch to the M model	68.8% (MVA)
1–2 Nov	YOLOv8-M on Drone2021, 10 epochs (Drone2021 re-downloaded)	44.6% (Drone)
2–4 Nov	YOLOv8-M: baseline and three data strategies, 30 epochs each	66.9–79.3% (MVA)
4 Nov	TTA, SAHI, merging rules, GFL ensemble, error analysis	84.5% (MVA)
4–5 Nov	Public-test inference (9,699 frames)	submission file
5–6 Nov	Paper, code and weight release	–

5.4.1 Pipeline Verification

Before any training, the inference and evaluation code was checked against a published number. The GFL checkpoint released with the challenge baseline [4, 17], which is initialised from COCO-pretrained weights and processes full 3840×2160 frames, reproduces its reported 47.0% AP and 88.9% AP50 (with flip TTA) on the same 879 validation frames with our code. Our own models are trained without any downloaded weights, a constraint fixed at the start of the project; this explains most of the gap to that reference and is the main reason our numbers should not be compared with challenge entries. A GFL trained from scratch on MVA alone reached only 6.1% AP50 after one epoch, which led us to pretrain both detectors on Drone2021.

5.4.2 Challenges and How They Were Addressed

A new GPU with old libraries. The Blackwell GPU (compute capability 12.0) requires PyTorch 2.7 with CUDA 12.8, for which no binary build of MMCV 1.6, the basis of the GFL code, exists. We used the pure-Python MMCV with a small compatibility layer that replaces its CUDA operators by torchvision NMS, and verified it on six NMS test cases of 1,200 boxes against the original CPU implementation (identical outputs); on two test frames GPU inference was about 19 times faster than the CPU fallback. Data-loader worker processes hung on Windows and were disabled for GFL inference.

BatchNorm with a batch of one. Fitting 2560×1440 inputs into 8 GB forced a physical batch of one with gradient accumulation. Batch normalisation then normalises each image with its own statistics during training but uses running averages

at test time. For GFL this was fixed by using per-image normalisation in both training and inference. The first YOLO pretraining run suffered from the same mismatch without it being visible in the loss: its validation AP50 fell to 3.4% at epoch 4, while re-evaluating the same weights with per-image statistics on 500 frames gave 35–37%. We therefore rewrote the YOLO recipe to train on native-resolution 1280×736 crops with real batches (8 for YOLOv8-S, 6 for YOLOv8-M), which is also how slicing entered the study. For YOLOv8-M a batch of 8 filled the GPU memory and ran at 14.3 images/s; a batch of 6 ran at 18.6 images/s. Mixed-precision overflow occurred 37 times during GFL training (10 on Drone2021, 27 on MVA); each time the step was retried with a halved loss scale.

Throughput and reliability of a laptop. Decoding 4K JPEG frames limited YOLO training to 12.3 images/s; decoding and crop generation in a thread pool raised this to 25.8 images/s. After a restart, Windows power management moved the data-loading threads to efficiency cores and throughput fell to about 10 images/s, until a wrapper disabled power throttling for the training process. Training was interrupted by a GPU driver reset, by an application-hang termination during validation and by planned shutdowns. All trainers therefore save atomic checkpoints (model, optimiser, scaler, random-number state and position in the epoch), and resumption was verified to give exactly the same result as uninterrupted training; the hash of the training code is stored with each run, so that a resumed run cannot silently use modified code. Public-test inference was first stopped after about 3,700 of 9,699 frames and lost; it was rewritten to save results in chunks of 500 frames.

Data. One exactly duplicated box was removed from the validation labels. Drone2021 and MVA use different box conventions (Section 4), and Drone2021 pretraining overfitted after 4 epochs for YOLOv8-S and 7 for YOLOv8-M; a second pretraining phase with stronger augmentation did not improve the starting point and was not used. Most misses of 32–64 px birds by YOLOv8-S came from three frames with dense flocks (124 of 134 misses). The 63.5 GB of Drone2021 images had to be deleted for disk space and downloaded again when we moved from YOLOv8-S to YOLOv8-M.

Compute budget and abandoned branches. With one GPU every experiment delayed the next, so the plan changed several times. A first round of YOLO and Swin Transformer experiments (9–28 October; native and upscaled crops, NWD loss, DySample upsampling) had been run with a batch of one; on 29 October it was discarded and the YOLO study was restarted under one controlled protocol, so none of its results are used here. YOLOv8-S was stopped after one MVA strategy in favour of YOLOv8-M. A YOLOv8-M variant with an additional high-resolution P2 head and a combined recipe (slicing, copy-paste and colour augmentation in one run) were prepared and tested on the CPU; the combined recipe was started on free cloud T4 GPUs, but both were dropped to meet the deadline. Instead, the remaining time went into inference-time methods and the heterogeneous ensemble, which need no further training. A missing plain YOLOv8-M baseline was noticed late and trained before the ablation was considered complete.

5.4.3 Lessons

Three lessons transfer to similar projects. First, verify the evaluation code against a published checkpoint before training anything. Second, with small batches, check batch normalisation explicitly: a train/test mismatch can look like overfitting. Third, on a single consumer GPU, resumable training with verified exact resumption and pinned code is worth its cost, because interruptions are the rule rather than the exception.

5.5 Limitations

First, slicing changes two factors at once: it covers every bird in every epoch and it produces 3.6 times more samples, hence more optimiser updates, than the other branches. Our ablation does not separate the two; a compute-matched baseline is left for future work. Second, all comparisons use a single seed, and the GFL checkpoint was chosen on the validation set (about 0.7 AP50 above its last epoch), which slightly favours GFL and the ensemble. Third, validation frames come from the same videos as training frames, so scene-level generalisation is not measured, and the public test score was not available to us at the time of writing.

6 Conclusion

We studied where resolution should be preserved for detecting tiny birds in 4K frames. Exhaustive native-resolution tile sampling during training is the most effective single change. Tiled inference with SAHI is redundant, and harmful, once frames are processed without downscaling; its loss is explained by scale mismatch and tile artefacts. Flip TTA merged with WBF and a heterogeneous WBF ensemble with GFL are complementary, raising AP50 from 70.2% to 84.5%, AP75 from 25.5% to 45.8% and AR100 from 43.9% to 53.4%, because the two detectors are strong on different bird sizes. Future work will separate the effects of tile coverage and update count in slicing, and target camouflaged birds, which dominate the remaining errors.

Author Contributions. Ngô Ngọc Trường Sơn: idea, implementation, all experiments, analysis and writing. Nguyễn Gia Phúc: [to be completed by the authors]. Vũ Quốc Đạt: [to be completed by the authors]. Phan Thị Thu Hồng: supervision. All authors read and approved the paper.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Competing Interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process. During the preparation of this work, the authors used generative AI tools to assist with writing code and to improve the grammar and wording of the manuscript. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Data and Code Availability. Code, configurations, per-image results and trained weights are available at https://github.com/sotbatchien123/sod4sb-small-bird-detection. The MVA 2023 SOD4SB and Drone2021 images are not redistributed and can be obtained from the challenge organisers.

References

[1] Fujii, S., Akita, K., Ukita, N.: Distant bird detection for safe drone flight and its dataset. In: Proc. Int. Conf. Machine Vision Applications (MVA) (2021)

[2] Sun, H., Wang, Y., Cai, X., Wang, P., Huang, Z., Li, D., Shao, Y., Wang, S.: AirBirds: A large-scale challenging dataset for bird strike prevention in real-world airports. In: Proc. Asian Conf. Computer Vision (ACCV) (2022)

[3] Yoshihashi, R., Kawakami, R., Iida, M., Naemura, T.: Bird detection and species classification with time-lapse images around a wind farm: Dataset construction and evaluation. Wind Energy **20**(12), 1983–1995 (2017)

[4] Kondo, Y., Ukita, N., *et al.*: MVA2023 small object detection challenge for spotting birds: Dataset, methods, and results. In: Proc. Int. Conf. Machine Vision Applications (MVA) (2023)

[5] Ghiasi, G., Cui, Y., Srinivas, A., Qian, R., Lin, T.-Y., Cubuk, E.D., Le, Q.V., Zoph, B.: Simple copy-paste is a strong data augmentation method for instance segmentation. In: Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pp. 2918–2928 (2021)

[6] Akyon, F.C., Altinuc, S.O., Temizel, A.: Slicing aided hyper inference and fine-tuning for small object detection. In: Proc. IEEE Int. Conf. Image Processing (ICIP), pp. 966–970 (2022)

[7] Solovyev, R., Wang, W., Gabruseva, T.: Weighted boxes fusion: Ensembling boxes from different object detection models. Image and Vision Computing **107**, 104117 (2021)

[8] Hou, H.-Y., Shen, M.-Y., Hsu, C.-C., Huang, E.-M., Huang, Y.-C., Xia, Y.-C., Wang, C.-Y., Lee, C.-Y.: Ensemble fusion for small object detection. In: Proc. Int. Conf. Machine Vision Applications (MVA) (2023)

[9] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.3.40 (2023)

[10] Li, X., Wang, W., Wu, L., Chen, S., Hu, X., Li, J., Tang, J., Yang, J.: Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In: Advances in Neural Information Processing Systems (NeurIPS) (2020)

[11] Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: Towards real-time object detection with region proposal networks. In: Advances in Neural Information Processing Systems (NeurIPS) (2015)

[12] Cai, Z., Vasconcelos, N.: Cascade R-CNN: Delving into high quality object detection. In: Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pp. 6154–6162 (2018)

[13] Zhou, X., Wang, D., Krähenbühl, P.: Objects as points. arXiv preprint arXiv:1904.07850 (2019)

[14] Zhang, H., Wang, Y., Dayoub, F., Sünderhauf, N.: VarifocalNet: An IoU-aware dense object detector. In: Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pp. 8514–8523 (2021)

[15] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: Proc. European Conf. Computer Vision (ECCV), pp. 740–755 (2014)

[16] Wang, J., Xu, C., Yang, W., Yu, L.: A normalized Gaussian Wasserstein distance for tiny object detection. arXiv preprint arXiv:2110.13389 (2021)

[17] Shinya, Y.: USB: Universal-scale object detection benchmark. In: Proc. British Machine Vision Conf. (BMVC) (2022)

[18] Shinya, Y.: BandRe: Rethinking band-pass filters for scale-wise object detection evaluation. arXiv preprint arXiv:2307.11748 (2023)

[19] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016)

[20] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In: Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), pp. 2117–2125 (2017)

[21] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: Proc. Int. Conf. Learning Representations (ICLR) (2019)